

0x01 搭和结构

C 语言内存布局 · 逆工程基础篇

学习点整理 · 前置知识清单与大量 实例

本书最要底槽槽代码

```
* ( _ B Y T E * ) ( * ( _ Q W O R D * ) ( a 1 + 8 2 0 8 ) + 4 1 2 0 L L )  
( c o n s t c h a r * ) ( a 1 + 4 1 1 2 )
```

这两行代码从某用到三个概念 字节 搭和 结构 本书把它们拆成 7 个部分
0 前置清单 + 第 1 ~ 6 章内容, 从进与十六进开始一路推能读懂上面两行并
且能推出对应的 C 结构体定义

怎么用这份讲义

先看第 0 章前置清单能够跳过第 1 ~ 4 章是也 必须用代码验证第 5 章是篇的
落点把逆向代码录开第 6 章自题做完答案 每一节配有编译的例子, 建议
全部跑一遍

通过学 C 的基本语法 变量、i f、f o r、函数, 但系统对搭与内存
目前点能懂 I D A / G h i d r a 反编译结构体与搭解用码

目 录

第0章 前置知识	4
0. 1 必须缺一项都会挂	4
0. 2 加层有好处	4
0. 3 第0章的个实验实验	5
实验1:看清楚一个整数的字节	5
实验2:sizeof 与地址度	5
实验3 :把地址当索引	6
第1章 内存与字节	7
1. 1 内存模型 带符号的一排格子	7
1. 2 数宽度与十六进表示	7
越容易看错的地方 0x 2010 与 8 208	8
1. 3 小端与大端 字节数怎么摆	8
为什么必须在意字节序	9
实验4:用union 直验字节序	9
第2章 类型与内存局	10
2. 1 类型的重 含义	10
2. 2 常见类型对照	10
2. 3 C 语言内建类型在内存里的样子	11
整数直接字节摆放	11
浮数IEEE 754 格式不能猜	11
指针存储就是一个地址	12
2. 4 结构体把变量 打包	12
第3章 指针概念	13
3. 1 声明 取地址 解用	13
3. 2 指针的大小与它的内容	13
3. 3 指针运算 + 1 到底多少	14
实验7:用地址验证指针算术	14
3. 4 指针与数组 各有其真面目	15
实验8 :数组退与下标运算	16
3. 5 多级指针	17
实验9 :二级指针的内存存	17

3 . 6 指与 c-const	1 8
3 . 7 函数	1 8
函数数组 模拟表	1 9
3 . 8 指与字符串	1 9
第4 章 结构体与填充	2 0
4. 1 为什么要齐	2 0
4. 2 一个必须已推一 遍结构体	2 0
4. 3 调整顺序能省内存	2 1
4. 4 实验打真偏移大小	2 2
4. 5 嵌套结构与数组成员	2 3
4. 5. 1 实验 13 :验证偏移	2 3
4. 6 位域与紧凑结构体(解阿)	2 4
第5 章 逆代码器解析	2 5
5. 1 目标码与它的含义	2 5
5. 2 第一步 把数转换成十六进制	2 5
5. 3 第二步 先出偏移表 再推结构体	2 6
5. 3 . 1 为什么 0x 2 010 是_ Q WORD 而不是_ D WORD	2 6
5. 4 第三步 逐条解第一个表达式	2 6
5. 5 第四步 解第二个表达式	2 7
5. 6 第五步 还原结构体定义	2 8
5. 7 从编译器再确认一次	2 9
5. 8 完整运行例子:自造一个简单结构体	3 0
第6 章 自测可能往你抛的	3 2
6. 1 二十个高频疑点	3 2
6. 2 逐解答	3 2
Q 1 为什么指针还要类型	3 2
Q 2 声明里的 * 与表达式里的 *	3 2
Q 3 p+ 1 与 (c h-a-r*)p+ 1	3 3
Q 4 s i ze of 数组形参	3 3
Q 5 2[a] 为什么合法	3 3
Q 6 数组名为什么不能值	3 3

Q 7	&a 与 &a[0]	3 3
Q 8	空理 是么	3 3
Q 9	为什么不等成员和	3 3
Q 10	换顺序为什么变小	3 3
Q 11	为什么 Windows 上 long 还是 4 字节	3 3
Q 12	_QWORD 与 _DWORD 读同一地址	3 4
Q 13	为什么是强转	3 4
Q 14	(*f_p)() 与 f_p()	3 4
Q 15	字符串 为什么需要尾 0	3 4
Q 16	const char *p 限制符	3 4
Q 17	为什么 int 成员偏移 1	3 4
Q 18	mov 与 lea	3 4
Q 19	为什么减六进制	3 4
Q 20	成员名还是偏移	3 4
6. 3	动手练习	3 5
练习 1		3 5
练习 2		3 5
练习 3		3 5
练习 4		3 5
练习 5		3 5
6. 4	练习答案	3 6
答案 1		3 6
答案 2		3 6
答案 3		3 6
答案 4		3 6
答案 5		3 7
6. 5	学习建议	3 7
6. 6	相对表	3 7
6. 7	一句话总结	3 8

0 前置知识

这一章不是可看不看的书而是一张检查表。搭桥以难绝大多数时候不是搭桥本身而是它赖的下层知识。不知道一个字里放什么、不知道地址哪来、不知道类型读几个字节、请逐行评论做不到按目录编号先去。

0.1 必须快速会(挂)

#	能顶	自然能顶这些吗
1	<code>0x2010</code> / <code>0x208</code> / <code>0x1018</code>	看到 <code>0x2010</code> 能立刻说出是进 8 2 08 ;看到 <code>4120</code> 能说出 <code>0x 1018</code>
2	<code>byte</code> / <code>bit</code>	知道 1 字节 = 8 位 = 2 个十六进制数 <code>0x FF</code> 是多少进制
3	<code>"00000000"</code>	能把一块内存想象成一排格子,每个格子 1 字节 8 位
4	<code>x86</code> / <code>x64</code>	知道 <code>x86</code> 用 4 字节地址 <code>x64</code> 用 8 字节地址知道地址就是符号整数
5	<code>0x11223344</code> / <code>44332211</code>	能出 <code>0x11223344</code> 在内存里按端是 <code>44332211</code>
6	<code>C</code> / <code>sizeof</code>	能用 <code>sizeof</code> 打印 <code>char/short/int/long/long long</code> / 搭桥的小并桥
7	<code>int a = 1; printf("%p", (void *)&a);</code>	能出 <code>int a = 1; printf("%p", (void *)&a);</code> 并解释输出含义
8	<code>C</code> / <code>main</code> / <code>gcc</code> / <code>clang</code> / <code>MSVC</code>	能写 <code>main</code> 函数用 <code>printf</code> 、用 <code>gcc/clang/MSVC</code> 编译运行
9	<code>int a[5]</code>	知道 <code>int a[5]</code> 的 5 个元素在内存里是连续的
10	<code>gdb</code> / <code>x64dbg</code>	能在 <code>gdb</code> 或 <code>x64dbg</code> 里给变量下断点查看内存

表 0-1 前置知识必须

0.2 加层有更快

- Hex 编辑器 十六进制编辑器能打开一个二进制文件并任意偏移
- 汇编入门 能看懂 `mov` / `lea` / `add` / `call` 这些指令知道寄存器大概是什么的
- PE 结构 知道 `EXE` / `DLL` / `PE` 段(`text` / `.data` / `.rdata`)大概区别
- 栈与调用约定 知道栈向下增长 函数调用有栈帧
- 字符串 知道 `ASCII` 码字符 `A` 是 `0x 41`, 数字 `0` 是 `0x 30`, 字符串以 `0` 结尾

0.3 第0章的个实验实验

下面三个程序很短,但它们是所有内容的显微镜。请务必跑一遍,并且把输出和你里的预期对照。

实验1 :看清楚一个整数的四个字节

```
e xp1 .c — — 观察程序
#include <stdio.h>

int main(void)
{
    int a = 0x11223344; /* 11 22 33 44 */
    unsigned char *p = (unsigned char*)&a; /* ? int * ? 1 ?????? */

    for (int i = 0; i < 4; i++)
        printf("t?f %d ??? ( ?? + %,d) i,=p[i])x %02X \n ",
               i, p[i]);

    return 0;
}
```

典型输出: 0 6/x 64 小端:

```
? 0 ??? ( ?? + 0 ) = 0x44
? 1 ??? ( ?? + 1 ) = 0x33
? 2 ??? ( ?? + 2 ) = 0x22
? 3 ??? ( ?? + 3 ) = 0x11
```

这个实验说明了什么

一是变量在内存里就是连续的字节;二是低地址放低字节(小端);三是同一个地址用不同类型看到的宽度不同。这三点是计算机的常识。

实验2:sizeof 与地址宽度

```

e xp2.c  — —  英尺族

#include <stdio.h>

int main(void)
{
    printf("tcfh ar      , s i z %cchuf))m "
    printf("tsfh o r t    , s i z %schuf))m t"
    printf("tifn t        , s i z %izon)) n "
    printf("tlfo n g        , s i z %lzoaf))g "
    printf("tlfo n g l o n g, s i z %lzoafnlgo ))g
    printf("tffl o a t    , s i z %fzoluf))m t"
    printf("tdfo u b l e      , s i z %dzoafu))j e
    printf("tvfo i d *       , s i z %vzoafi*));"
    printf("tcfh ar *        , s i z %cchufa*));"
    printf("tifn t *         , s i z %izonft))n "
    return
}

```

在 64 位 Windows (MSVC / MingW64) 上, long 是 4 字节在 64 位 Linux 上, long 是 8 字节这是跨平台时最容易踩的坑之一。

实验 3: 把地址当数打印

```

e xp3.c  — —  地址是数

#include <stdio.h>
#include <stdint.h>

int g_val @ 0x1234 /* ????, ??? .data */

int main(void)
{
    int oca 0x5678 /* ????, ??? */
    printf("tgf_ v a l u e  ?,? (V o =i*)%8g p v m))l u e
    printf("tlfo c a l    ?,? (V o =i*)%8l p c))l
    printf("tmfa i n      ?,? (V o =i*)%8m p i))m "
    /* ?????????????, ?????????? */
    printf("t?f? ? ? ?      , = 0x%11X \n "
        (unsigned long)(gint p)&g_r_vta))l u e
    return
}

```

为什么这很重要

逆时看到的 `a 1 + 8 2 08` 这种写法本质是 某对象的地址 + 偏移; 能受地址就是字可加减, 不会被当作运算量

1 内存与字节序

本章所有讨论的前提是内存长什么样 一个值怎么被拆成字节 字节按什么顺序放

1.1 内存模型 数据的排布

把内存想象成一行格子,每个格子恰好1字节8位,每个格子有一个编号,这个编号就是地址。地址从0开始逐一递增,32位的编号范围是0x 00000000 ~ 0x FFFFFFFF,64位则是0x 0000000000000000 ~ 0x FFFFFFFFFFFFFFFF (实际可寻址范围)。

图1-1 内存是排布的样子

```
?? (1 ??)
0x00002000  U A A A A A A A A A ;
3 0x44 3 <- ?? a ?? 0 ???
0x00002001  A A A A A A A A A A '
3 0x33 3 <- ? 1 ???
0x00002002  A A A A A A A A A A '
3 0x22 3 <- ? 2 ???
0x00002003  A A A A A A A A A A '
3 0x11 3 <- ? 3 ???
0x00002004  A A A A A A A A A A U

?: int a = 0x11223344; ??? 0x2000~0x2003 ? 4 ???
a ? " ? ? " ? ? ? ? ? ? ? 0x2000.
```

1.2 数据宽度与十六进制表示

十进制与二进制 字是一一对应的,这有什么意义 几乎不用二进制

十进制位数	二进制位数	字数	C 类型	x86 常见	IDA 类名	取值范围示例
2	8	1	char / unsigned char		BYTE	0x 00 ~ 0x FF
4	16	2	short		WORD	0x 0000 ~ 0x FFFF
8	32	4	int / long		DWORD	0x 00000000 ~ 0x FFFFFFFF
16	64	8	long long / 指针	x64	QWORD	0x 0 ~ 0x FFFFFFFFFFFFFFFF

表1-1 十进制数与类型宽度关系

为什么必须注意序

两种常量的结果

- **看内存值** 在内存里看到 44 3 3 2 2 11, 要立刻反应这是一个 int, 值是 0x 112 23 3 44"。
- **反编译与对照** 在反编译看到 mov eax, 443 3 2 2 11h, 而内存里是 11 2 2 3 3 44, 要这是同一数据的两种角度

实验4: 用 union 直观显示序

e xp4.c — — u n i o n 是看同一块内存解释器出具

```
#include <stdio.h>
#include <stdint.h>

typedef union
{
    uint32_t w; /* 4 bytes */
    uint8_t b[4]; /* 4 bytes */
} WordBytes;

int main(void)
{
    WordBytes u;
    u.w = 0x11223344;

    printf("0x%x = 0x%x\n", u.w, u.w);
    printf("b[0] = %02X b[1] = %02X b[2] = %02X b[3] = %02X\n",
        u.b[0], u.b[1], u.b[2], u.b[3]);

    if(u.b[0] == 0x44)
        printf("t=f> 0x44 (0x44); 0x44\n");
    else
        printf("t=f> 0x44; 0x44\n");
    return 0;
}
```

union 与 struct 的区别

struct 的成员是依序排开, 大约各成员都要占; union 的成员是全重叠在同一处, 大等最大成员。逆序中 union 常出现在解析寄存器定义和类型定义 (type punning) 里。

2 类型与内存布局

指针的类型决定了它指向什么，这一章把这件事一次说透

2.1 指针的意义

含义	说明	例子
读一个字节	从该地址取8位数据	<code>char*</code> 取1字节 <code>int*</code> 取4字节 <code>WORD*</code> 取8字节
解释这些字节	同样4字节可当 <code>int</code> ，也可当 <code>float</code> ，也可当4个 <code>char</code>	0x3F800000 当 <code>float</code> 是1.0f，当 <code>int</code> 是1065353216
加1走多远	指针+1移动的是一个元素的数据	<code>char*</code> +1 走1字节 <code>int*</code> +1 走4字节 <code>WORD*</code> +1 走8字节

表2-1 指针类型做的事

这最容易混淆的三条

很多人以为指针就是一个地址，于是不知道为什么区分 `char*` 和 `int*`。地址相同但以该地址为起点读多少字节，以及读多少倍由编译器决定，反编译里那些把同一个地址反复转成不同类型的写法正是在切换这种意义。

2.2 常见类型尺寸对照

类型	x86 (32位)	x64 Windows (LLP64)	x64 Linux (LP64)	说明
<code>char</code> / <code>_BYTE</code>	1	1	1	最小单位
<code>short</code> / <code>_WORD</code>	2	2	2	短整型
<code>int</code> / <code>_DWORD</code>	4	4	4	最常用整型
<code>long</code>	4	4	8	平台相关
<code>long long</code>	8	8	8	64位整型
<code>float</code>	4	4	4	单精度
<code>double</code>	8	8	8	双精度
<code>__int64</code> / <code>_QWORD</code>	8 (x64)	8	8	x64下翻倍
<code>size_t</code>	4	8	8	跟踪宽度

表2-2 同一类型在不同平台的大小，重点看 `long` 与 `指针`

平差带来的真结果

一个结构在 32 位程序里可能是 24 字节在 64 位程序里变成 32 字节成员也会变逆时针
第一步必须先确认目标的数据看 P E 头寄存器宽度, 否则算出来的偏移一定是错的

2.3 C 语言类型在内存的样子

整数直接字节摆放

```
int      a = 0x11223344;      /* 4 ?? */
short    s = 0x1122;          /* 2 ?? */
char      c = 0x11;            /* 1 ?? */
long long q = 0x1122334455667788LL; /* 8 ?? */

?????? ( ????? ):
a : 44 33 22 11
s : 22 11
c : 11
q : 88 77 66 55 44 33 22 11
```

浮数 IEEE 754 格式不能猜

float 的 4 个字节是符号 1 位 + 指数 8 位 + 尾数 23 位, double 是 1 + 11 + 52".
0x3F800000 就是 1.0f, 看到 0x40490FDB 就是 3.1415927f.

exp5.c — — 浮点格式

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    float f = 1.0f;
    unsigned char *p;
    mem(&p, &f, 4);
    printf("f = 1.0f, b = %08X\n", *p); /* 0x3F800000 */

    f = 3.1415927f;
    mem(&p, &f, 4);
    printf("f = 3.1415927f, b = %08X\n", *p); /* 0x40490FDB */

    double d = 1.0;
    unsigned long long *p;
    mem(&p, &d, 8);
    printf("d = 1.0, b = %016X\n", *p); /* 0x3FF0000000000000 */
    return 0;
}
```

逆推地址

在内存看到 0x 3 F8 00000 附近的数很像是常量 而不是数 看到 0x 42 F60000 就猜到 123 . 0f 。这类数能能助快还原

指针存的是一个地址

```
int a = 10;
int *p = &a;

32 ? ? ? ? ( ? ? a ? 0x0019FF2C ) :
&a = 0x0019FF2C ? ? 4 ? ? = 0A 00 00 00 ( a ? ? 10 )
&p = 0x0019FF20 ? ? 4 ? ? = 2C FF 19 00 ( p ? ? = a ? )

64 ? ? ? ? ( ? ? a ? 0x0000000065FE1C ) :
&a = 0x0000000065FE1C 4 ? ? = 0A 00 00 00
&p = 0x0000000065FE10 8 ? ? = 1C FE 65 00 00 00 00

? ? : p ? ? ? ? ? ? , ? ? ? ? ( 4 ? 8 ? ? ) ,
? ? ? ? ? a ? ? ? .
```

2.4 结构体多个变量

结构体就是把若干个变量按顺序排列，它是编程中最重要的类型因为它直接对应对象和偏表。

结构体声明是有顺序

```
typedef union P o i n t
{
    int x; /* ? ? + 0 */
    int y; /* ? ? + 4 */
} P o i n t

typedef struct P l a y e r
{
    int id /* ? ? + 0 */
    int hp /* ? ? + 4 */
    P o i n t p o s ; /* ? ? + 8 : ? ? ? ? ? ? ? ? ? ? */
    char name[16]; /* ? ? + 16 */
    void *w e a p o n /* ? ? + 32 ( x 6 4 ? ? ? ? ? ? + 3 2 ) */
} P l a y e r
```

各类型尺寸与"对齐值"的关系是结构体的第4章专门讲padding与对齐这里只需要先建立对结构体成员声明顺序敏感偏量由成员大小和对齐共同决定

3 指针概念

这一章重点讲指针，每个小节配一个运行例子，并且尽量给出内存图 + 打印输出 + 逆推里长什么样

3.1 声明取址解用

三个最操作

```
i n t a = 1 0
i n t * p = &a;    /* p 指向 a 的地址 */

p r i n t f ("t %d \n a);    /* 1 0      - - 指向? * /
p r i n t f ("t %d \n *p);    /* 1 0      - - 指向?, ? p 指向? * /
*p = 2 0    /* 指向? a = 2 0 */
```

写法	读法	含义
i n t * p ;	p 是一个指向 i n t 的指针	声明 p 用来装 i n t 的地址
& a	取 a 的地址	结果是 i n t *
* p	解用 p	取出 p 指向的 4 个字节并按 i n t 解释
p	指针自身值	就是一个地址字
& p	取变量 p 的地址	类型是 i n t * * , 即二级指针

表 3 - 1 指针相关写法含义

最容易的两个符号

声明里的 * 属于类型 (i n t * 是一个类型, 表达式里的 * 是解用运算符。同理 & 在表达式里是 "取址", 在声明里是引用 (C + +) 或参与两数运算。读代码时判断这个 * 是在类型位置还是在表达式位置能避免一大困惑

3.2 指针的大小与它的内容

e xp6 .c — — 指针的内存

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    int *p = &a;

    printf("sizeof(p) = %d, sizeof(*p) = %d\n", sizeof(p), sizeof(*p));
    printf("tpf: %d, (void*)p = %p\n", *p, (void*)p);
    printf("t&fp: %p, (void*)&p = %p\n", &p, (void*)&p);
    printf("t*fp: %d, *p = %d\n", *p, *p);

    /* ... */
    printf("tpf: %d, (void*)p = %p\n", *p, (void*)p);
    return 0;
}
```

一个反直觉的事实

指针变量本身也有地址，所以二级指针才定义为 `int **pp = &p;`，表示 `pp` 指向装着 `a` 的地址的 8 字节格子。逆过来，`*(a + 48)` 这类写法就是在结构体里取出一个指针。

3.3 指针 + 1 到底多少

这是初学者常犯的地方，指针 + 1 加的不是 1 个字节而是 1 个元素的字节数

```
?? base = 0x1000, ??? char=1, short=2, int=4, long=8

char *pc = (char *)base; pc + 1 = 0x1001
short *ps = (short *)base; ps + 1 = 0x1002
int *pi = (int *)base; pi + 1 = 0x1004
long long *pq = (long long *)base; pq + 1 = 0x1008

?: ?? = base + n * sizeof(*p)
n ????, ?? p - 1 ??????
```

实验 7: 用地址验证算术

```

e xp7.c  — —  同个地址 不同类型 +1  走同距离

#include <stdio.h>

int main(void)
{
    int arr[4] = { 0x1111, 0x2222, 0x3333, 0x4444 };
    int *p = arr;
    char *pc = (char*)p;

    printf("arr      , (v o %p)\n", p);
    printf("t&fa r r [ 1 ], (v o %p)&a r[1]);
    printf("tpfi + 1 = %p , (v b i*(p i+d1),? ? \n "
           "(l o nlgo )(c h a*)(p i+ 1) - (c h a*)p i);
    printf("tpfc + 1 = %p , (v b i*(p i+d1),? ? \n "
           "(l o nlgo )(p c+ 1) - p i);

    printf("t*fp i      = , *p i;% 0 8 X \n "
           "t*f( p i + 1 ) = , *(p i+ 1);X \n "

    /* ? char* ???? int ???? (??) */
    for (int i = 0; i < 4; i++)
        printf("t?f %d ? int ??,? i, p i *04}% 0 2 X \n "
               ret u, n
}

```

这是为什么反编译出来是非类型

当需要按1 字节扫描一块内存时会写成 (char*)p + i ; 需要按4 字节读一个整数时就写 *(_ D W O R D *)p。强类型不是为了改变内存而是为了改变读取与步长。

3.4 堆与数组名等价

在大多数情况下，数组会退化成指向元素的指针。这解释了为什么数组名在表达式中通常会被当作指针来处理。

数组与指针的关系

```
int a[5] = { 1 0 2 0 3 0 4 0 5 0};

/*      ? ? ? ? ? ? ? ? ? ?      */
a[2]      == *(a + 2)    == *(2 + a)    == 2[a]

/*      ? ? ? ? ? :      */
a[2]  <=> *(a + 2)          ????? " ? ? ? ? ? ? ? "
      <=> *(2 + a)          ?????
      <=> 2[a]              ?? 2[a] ??(?????)

/*      ? ? ? ? ?      */
a      == &a[0]              ?????????????
a + 2   == &a[2]
*(a + 2) == a[2]
```

两个必须搞清楚

`int a[5];` `sizeof(a)` 是 20 个字节, 而在函数里 `int a[]` 会退化成 `sizeof` 变成 `int` 就是数组作为函数参数会丢失信息的原因也是这里 函数名写成 `(int *a, int n)` 的由来

实验 8 : 数组退化为下标运算

```
exp8.c — — 数组退化为下标运算

#include <stdio.h>

void isdh (int arr[], int n) /*      ? ?      arr      ? ? ? ? ?      */
{
    printf("sizeof (arr) =, %zu\n", sizeof(arr));
    for (int i = 0; i < n; i++)
        printf("arr[%d] = %d * (arr + %d) = %d * (arr + %d) = %d\n",
               i, arr[i], i, *(arr + i), i, *(i + arr));
}

int main (void)
{
    int a[5] = { 1 0 2 0 3 0 4 0 5 0};
    printf("sizeof (a) =, %zu\n", sizeof(a));
    printf("a[2] = %d 2[a] = %d * (a + 2) = %d * (2 + a) = %d\n",
           a[2], 2[a], *(a + 2), *(2 + a));
    show(5);
    return 0;
}
```

3.5 多级指针

多级指针说白了就是指的指针：每一层多一次引用就多一次内存跳转

多级指针

```
int a = 42;
int *p = &a;          /* ?? : p -> a */
int **p = &p;          /* ?? : p p -> p -> a */

printf("t%fd \ a);"   /* 42 */
printf("t%fd \ *p);"   /* 42 */
printf("t%fd \ **p);"  /* 42 - - ??? */

/* ??? */
/* a : int ? = 42 */
/* p : int * ? = &a */
/* p p : int ** ? = &p */
/* &p p : int *** ? = p p ??? */
```

实验 9: 二级指针的内存

e xp9.c — — 二级指针的内存

```
#include <stdio.h>

int main(void)
{
    int a = 42;
    int *p = &a;
    int **p = &p;

    printf("t&a = %p, (v%i*)&a, a);d \ n "
    printf("t&fp = %p p = (v%i*)&p, (v%i*)p, *p);% d \ n "
    printf("t&fp p = %p p p = %p, * p p = %p ** p p = %d \ n "
           (v%i*)&p (v%i*)p (v%i*)*p **p);

    **p = 10;0 /* ??? a */
    printf("t?f * * p p ??, a); = %d \ n "

    *p = NULL /* ??? p ?? */
    printf("t?f * p p ??, (v%i*)p);p \ n "
    return 0;
}
```

3.6 堆与 const

写法	谁修改	记忆诀
<code>const int</code>	不能通过p改值p自己指向	<code>const</code> 在* 左边值不可
<code>int const</code>	同上(等价)	同上
<code>int * const</code>	自己指向不可改可以通过p改	<code>const</code> 在* 右边指向不可
<code>const int</code> <code>const p;</code>	两者不可改	两者有都不可

表3 - 2 const 与堆的组合

逆理为什么常见 const char *

反编译里 `(const char *)a` 1 通常表示把a 1 当作字符串的地址。它不改任何内存只声明这里 开始是一串字符。看这个转换基本可以判断后续它传给 `strlen/sprintf/printf` 。

3.7 函数指针

函数在内存 有地址因此指向函数的指针存在 逆向函数指针常见虚表(vtable)、回调消息处理等。

e xp1 0.c — — 函数指针

```
#include <stdio.h>

int add(int a, int b) { return a+b; }
int sub(int a, int b) { return a-b; }

int main(void)
{
    int (*fp)(int, int) = add; /* fp 指向 add 函数，int 指向 int 类型 */

    printf("add: %d\n", (void *)add);
    printf("fp: %p\n", (void *)fp);
    printf("add(3, 4) = %d\n", add(3, 4));
    printf("(*fp)(3, 4) = %d\n", (*fp)(3, 4));

    fp = sub;
    printf("fp: %p\n", (void *)fp);
    return 0;
}
```

函数数组模型表

e xp1 1 .c — — 函数数组模型表

```
#include <stdio.h>
```

```
int o_pveon( f p u f"so p "; n r e t u0; }
```

```
int c_lvoos( e f p u f"sc l o); s e d t u0; }
```

```
int r_evad( f p u f"sr e "; d r e t u0; }
```

```
/* ? ? ? ? ? ? , ? ? ? ? ? ? ? ? */
```

```
int (*g_v t b)(v o) { o n _ o, poenn_c l o b _ e r }; a d
```

```
int ma (vno) d
```

```
{
```

```
fo (i n t = 0; i < 3; i++)
```

```
g_v t b( i ); e /* ? ? ? ? o p e n / c l o s e / r e a d */
```

```
/* ? ? ? ? ? ? 1 ? ? ? ? ? ? */
```

```
int (*f b)(v o) { g_v t b; l e
```

```
f ();
```

```
ret u0; n
```

```
}
```

逆置阿列函数

在x 64 上,如果看到c a l l qword ptr [ra x + 18 h] 这种调用,它里面的值,就是通过数计调用 + 18 h 就是表中第3 项(每8 字节0, 8, 10, 18)。把这张表在IDA 里还原成结构是逆向C++ 程序的关键一步

3.8 堆与字符串

字符串就是以0 结尾的数组

```
char s1[] = "hello"; /* 6 ? ? ? ? ? ( ? ? ? \ 0 ), ? ? ? ? */
```

```
char s2[] = "hello"; /* ? ? ? ? ? ? ? ? ? ? ? ? ? ? , ? ? ? ? */
```

```
char s3[] = s1; /* ? ? ? ? ? ? ? ? ? ? */
```

```
printf("t%fs\n", s1); /* hello */
```

```
printf("t%fc\n", s1[1]); /* e */
```

```
printf("t%fc\n", s1[1+1]); /* e - - ? ? */
```

```
printf("t?f? %z\n", (s1)); /* 5, ? ? ? ? \ 0 */
```

为什么反编译时写的是(const char *)

字符串在内存里没有长度,只有结尾的0x 00。所以处理字符串,只需要一个地址编译器就把它写成const char *。看到(const char *)(a 1 + 4112),就理解成"从a 1+ 4112 串以0 结尾的字符串。"

4 结构体与填充

这一章解决两个实际问题：给定一串偏量，还原结构体定义。要做到这一点必须理解 padding 填充和内存洞。

4.1 为什么需要齐

CPU 访问内存不是任意都能随便取的。多线路上，访问一个 4 字节整数时如果起始地址是 4 的数，硬件一次就能读完。否则要分成两次访问，拼接，甚至触发异常。

- **对齐值 (align)**：每类有一个对齐值，通常等于它自己的大小，但不超过平台的最大对齐。
- **对齐规则**：结构体成员必须在对齐值的整数倍偏址上，编译器为此插入字节。
- **填充 (padding)**：为对齐而插入的字节，里面垃圾值，读没有意义。

类型	大小	对齐值	地址必须是
char	1	1	任意地址
short	2	2	2 的倍数
int	4	4	4 的倍数
float	4	4	4 的倍数
long long	8	8	8 的倍数
double	8	8	8 的倍数
__x64__	8	8	8 的倍数

表 4-1 常见的对齐 (64 默认)

4.2 一个必须自推的结构体

sizeof (struct A) 是多少？先算看下面

```
struct A{
    char c;    /* 1 ?? */
    int i;     /* 4 ?? */
    short s;   /* 2 ?? */
    double d;  /* 8 ?? */
};
```

逐推：假设结构体本身按 8 字节对齐：

图4-1 struct A 的布局

屠填也要算

结构体大小必须得到最成员值的整倍, 这样数里 每行满起地址仍然满足要求, 所以 size of 往往是 8 的数

同样的员换个顺差8字节

21

逆时还有用

当你在逆时 发现一个结构体的大小而 2 4 时 而 3 2 , 不一定是错了, 很可能是你的声明顺序不同 或者用不同编译选项 判断你观察到的偏移, 而不是你觉得合理的顺序。

4.4 实验打印偏移大小

不要相信直接编译告诉你 of f s e t o f 是标准宏 必须掌握

e xp1 2.c — — o f f s e t o f 是最初版本

```
#include <stdio.h>
#include <stddef.h> /* offsetof */

typedef union MyData {
    int id; /* + 0 */
    char name[16]; /* + 4 */
    void *next; /* + 20 -> ??? + 24 (x64) */
    double score; /* + 32 */
} MyData;

int main(void) {
    printf("sizeof ( MyData ) is %d \n", sizeof(MyData));
    printf("offsetof ( id ), offset ( MyData ) is %d \n",
        offsetof(MyData, id));
    printf("offsetof ( name ), offset ( MyData ) is %d \n",
        offsetof(MyData, name));
    printf("offsetof ( next ), offset ( MyData ) is %d \n",
        offsetof(MyData, next));
    printf("offsetof ( score ), offset ( MyData ) is %d \n",
        offsetof(MyData, score));

    /* ??? */
    MyData d;
    printf("t&d = %p (char *) &d.id, - (char *) &d = %lld \n",
        (void *)&d, (long)&d - (char *)&d);
    printf("t f (char *) &d.next - (char *) &d = %lld \n",
        (long)&d - (char *)&d.next);
    return 0;
}
```

逆时的方法

反编译给你一堆偏移 (+ 0、+ 4、+ 0x 18、+ 0x 20)。在本地上写一个候选结构体用 of f s e t o f 打印成员的偏移, 两列表一对就能验证你的是否正确 这是最可靠的手段

4.5 结构体与数组的成员

成员偏移 = 外层偏移 + 内层偏移

```
typedef union Vte { 3
    float x, ty, z;          /* +0 +4 +8, ?? 12 */
} Vec3;

typedef struct Ent_t { ty
    int id;                  /* +0 */
    Vec3 pos;                /* +4 : ?? 12, ??? 0/4/8 */
    Vec3 vel;                /* +16 : ??? */
    char name[32];          /* +28 */
    int flags;               /* +60 */
} Entity_t;

/* : " " : */
/* e.pos.y -> &e + 4 + 4 = &e + 8 */
/* e.vel.z -> &e + 16 + 8 = &e + 24 */
/* e.name[i] -> &e + 28 + i */
/* e.flags -> &e + 60
```

4.5.1 实验 1 3: 验证偏移

exp1_3.c — 验证偏移

```
#include <stdio.h>
#include <stddef.h>

typedef union Vte { float x, ty, z; } Vec3;

typedef struct Ent_t { ty
    int id;
    Vec3 pos;
    Vec3 vel;
    char name[32];
    int flags;
} Entity_t;

int main(void)
{
    printf("sizeof (Vec3) , size (Vec3) %zu \n",
           sizeof (Vec3), sizeof (Vec3));
    printf("sizeof (Entity_t) , size (Entity_t) %zu \n",
           sizeof (Entity_t), sizeof (Entity_t));
    printf("offsetof (pos), offset (Entity_t) %zu \n",
           offsetof (Entity_t, pos));
    printf("offsetof (vel), offset (Entity_t) %zu \n",
           offsetof (Entity_t, vel));
    printf("offsetof (name), offset (Entity_t) %zu \n",
           offsetof (Entity_t, name));
    printf("offsetof (flags), offset (Entity_t) %zu \n",
           offsetof (Entity_t, flags));

    /* : pos.y : = offsetof (pos) + offsetof (Vec3, y) */
    printf("tpos.y : = %zu ( : %zu + 4 ) \n",
           offsetof (Entity_t, pos) + offsetof (Vec3, y),
           offsetof (Entity_t, pos));
    return 0;
}
```

4.6 位与紧结构体(解问)

位与 #pragma pack

```
/*  ????:???,????,??? */
typedef struct a{
    unsigned char: 1; /* ? 1 ? */
    unsigned char: 3; /* ? 3 ? */
    unsigned char: 4; /* ? 4 ? */
} Flag;

/*  ?????:???,???/???? */
#pragma pack(push, 1)
typedef struct b{
    char tag; /* +0 */
    int value; /* +1 ???!(????+4) */
    short rc; /* +5 */
} Packed;
#pragma pack(pop)
```

逆向看到奇怪偏移量 pack

观察到 int 成员偏移 1、5 这种非 4 倍位置可以确定结构使用了 pack (1) (即紧凑格式)。在 IDA 里 可以给结构设置 pack ed "属性"。

5 逆向代码逐层解

现在把所有知识结合起来, 彻底揭开逆向代码。这一章会建立偏移表 = 结构的应
关系, 再逐层剥开。

5.1 目标代码的含义

目标: 两行汇编代码

```
( * ( _ B Y T E * ) ( * ( _ Q W O R D * ) ( a 1 + 8 2 0 8 ) + 4 1 2 0 L L )  
( c o n s t c h a r * ) ( a 1 + 4 1 1 2 )
```

先结论再推导。这两行说明 `a_1` 指向一个很大的对象 (至少 123 28 字节); 在 `a_1 + 8 208` 处放
着一个指针 (8 字节, 所以是 `_ Q W O R D`); 该指针指向一个对象 (偏移 4120 处有一个
字节, 而 `a_1 + 4112` 处开始是一串字符 (字符串))。

一个非常关键的观察

这些偏移 (4112、8 208) 都是 16 的倍数, 而且彼此很远。这可不像是手写的结构体,
更像是若干个类型结构体顺序排列或者数组。8 208 - 4112 = 4096 = 0x 1000, 这个数字
太整了, 暗含一大片区域。

5.2 第步把数都换成十六进制

```
4 1 1 2      = 0 x 1 0 1 0  
4 1 2 0      = 0 x 1 0 1 8      ( 4 1 1 2 + 8 )  
8 2 0 8      = 0 x 2 0 1 0  
  
? ? ? ? ? ? :  
    * ( _ B Y T E * ) ( * ( _ Q W O R D * ) ( a 1 + 0 x 2 0 1 0 ) + 0 x 1 0 1 8 L L )  
    ( c o n s t c h a r * ) ( a 1 + 0 x 1 0 1 0 )  
  
? ? ? ? ? ? ? ? ? ? ? ? :  
    0 x 1 0 1 0      ?      0 x 1 0 1 8      ? ?      8      ? ?      - -      ? ? ? ? ? ? ? ? ? ? ? ?  
    0 x 2 0 1 0      ?      0 x 1 0 1 0      ? ? ?      - -      ? ? " ? ? ?      2      ? ? ? "  
    0 x 1 0 0 0      ? ? ? ? ? ? ( 4 0 9 6 ) , ? ?      0 x 1 0 1 0      ? ? " ?      +      ? ? " ? ? ?
```

为什么一定要换成十六进制

这里, 偏移都来自结构体布局, 而结构体布局由对齐决定。对齐值是 2 的幂, 所以偏移在
十六进制下呈现极规律的状态 (0x 1010、0x 1018、0x 2010、0x 2020...), 在进制转换规律
律。换成十六进制再找规律, 是逆向的基本功。

5.3 第二步 先翻偏移表再推结构

逆结构的思路是把代码里出现的所有偏移收集起来按谁只差一个类型大小分组然后猜测类型

观察到的偏移	推测类型	理由
+ 0 x 1 0 1 0	c h a r[?] 或指向字符串的段被截 c o n s t c h a r * , 是字符串起点	
+ 0 x 1 0 1 8	位于另一个象内部偏移 0x 1018	先用 _ Q W O R D , 再加 0x 1018
+ 0 x 2 0 1 0	_ Q W O R D 8 字节指针	被 _ Q W O R D * 解用说明这里存指针
+ 0 x 1 0 0 0	可能是一个结构数组	0x 1010 - 0x 1000 = 0x 10

表 5-1 从偏移推类型

5.3.1 为什么 0x2010 是 _ Q W O R D 而不是 _ D W O R D

这是最关键的一步推断代码是 * (_ Q W O R D *) (a 1 + 0x 2010) , 意思是从 a 1 + 0x 2010 开始 8 字节把这 8 个字节当成一个地址。如果里 其是 4 字节的 _ D W O R D , 编译结果不会是这样所以断定

- **推论 1** 这是一个 64 位程序否则就不会是 8 字节。
- **推论 2:** a 1 + 0x 2010 处放的是一个成员而不是数

逆中宽类型推理

_ B Y T E / _ W O R D / _ D W O R D / _ Q W O R D 直接按 1/2/4/8 字节看到 _ Q W O R D 位数想看到 _ D W O R D 就往 i n t 或 f l o a t "想 这是反编译里 提类型信息最快 最可靠的手段

5.4 第三步 逐层解第一个表式

面对表式从里面往外刷一次做一件事并且每一步都清楚现在是什么 值是什么。

从第五层解

```
??:  
  *(_ B Y *)(*(_ Q W O*)(@ 1+ 8 2 0 8 4 1 2 0 L L  
  
? 0 ? :a 1  
  ?? :??? (??), ??????  
  ? :???, ?? B a s e  
  
? 1 ? :a 1+ 8 2 0 8  
  ?? :B a s ? @ 8 2 0 ? B ?  
  ?? :??? (?????? " ? ? ) ? "  
  ? :B a s + e 0 x 2 0 1 0  
  
? 2 ? :*( _ Q W O*)(@ 1+ 8 2 0 8  
  ?? :? B a s 0 ex 2 0 ? 1 B ???, ??????  
  ?? : _ Q W O ? R D ? ? ? ? ? ? ? ? "  
  ? :?? P (????????)  
  
? 3 ? :*( _ Q W O*)(@ 1+ 8 2 0 8 4 1 2 0 L L  
  ?? :P 1???? 4 1 2 ? 0 ?  
  ?? :???? " 1 ? ? x, ?? ? ? ? ? _ B ' Y ? , E  
  ????????? " ? ? ? ? "  
  ? :P 1+ 0 x 1 0 1 8  
  
? 4 ? :*( _ B Y *)(*...)  
  ?? :? P 10 x 1 0 ? 1 B ???  
  ?? :u n s i g n e a d r  
  ? :???, ?? B  
  
???????? = ?? P 10 x 1 0 ? 1 B ???.
```

一个细节注意

第3层里 4120 后面跟着 L L 后缀 (ong l ong)。这说明编译器把偏移量放在了 64 位宽度上, 进一步印证是 64 位序, 同时也是这个数量级字节, 因为紧接着是 _ B Y T E 解用。

5.5 第五步拆解第二个表式

两层的解

```
??:  
(const char*)(a1+ 4 1 } 2  
  
? 0 ? : a 1  
?: Base  
  
? 1 ? : a1+ 4 1 1 2  
?: Base+0x1010  
?: ? Base? 0x10?1?0  
  
? 2 ? : (const char*)  
?: ?????? " ? ? ? ? ? ? "  
????,???" ? ? ? ? "  
  
???????? = ????? " . . ????,  
????? Base+0x10?1?00x0?0.
```

把两结果放在一起看规律

```
Base + 0x1010 -> ?????? (const char *)  
Base + 0x2010 -> ?????? (_WORD)  
  
0x2010 - 0x1010 = 0x1000 = 4096  
  
??? : ?????? / ???, ?? 0x1000 ?? .  
0x1010 ?? " ? 0x10 ?????, ??? 0x1000 ?????? " .  
??? : Base ??? " ? ", 0x1010 ????? 0 ??????,  
0x2010 ????? 1 ?????? (???? 0x1000) .
```

5.6 第五步还原结构体定义

把上面的写成C 代码 注意得到的结构体是解释 所有偏移的最结构不一定是作者写的, 这没问题

还原结构体与等价写法

```

/*  ?  a1  +  0x2010  ??????? ( ??  0x1018  ??????? )  */
typedef struct u_cstbobj {
    unsigned char *phead;
    unsigned char *pbody;
    /*  . . .  ???????  */
} SubObj;

/*  a1  ?????  */
typedef struct u_cmtainobj {
    unsigned char *phead;
    char *pbody;
    SubObj *pobj;
    /*  . . .  */
} MainObj;

/*  ??????????????????????:  */
unsigned char *phead = a1;
const char *pbody = a1;

```

这个过是游结构的编程

不要一上来就图完整结构 正确是 ① 收集代码出现的偏移 ② 换进制

找规律 ③ 用宽度判断类型 (BYTE = 1、_WORD = 2、_DWORD = 4、_QWORD = 8); ④ 把

区域声明成 unk_nown_ NNNN 字节数组 ⑤ 写结构体用 of f set of 验证是否吻合

⑥ 随着分析函数 逐步补充字节数组

5.7 从汇编角度再认识

反编译是编译器优化的结果 看汇编角度与层级是否正确

关键解读

```

; ???:  *(_BYTE *)(&_QWORD)(a1+82084120LL)
; ?? a1? rc, &4? Windb?W?s

mov  rax, x[r c+&01]0 h ; ? a 10 x 2 0?1?08 ???? -> rax
mov  z &a, xby tpet [r a+&01]8 h; ? r a+&01 0?118??,???? 3 2?
; ???? e a &,??????

ret n

; ???:  (const a*)(a1+41) 2
lea  rax, x[r c+&01]0 h ; ?????,????
ret n

```

指令

含义

对应C 代码

mov rax, [rcx+2] 从地址 rcx 开始 8 字节到 rax (cx 是地址) rax = *(_QWORD *) (a1 +

指令

含义

对应C 代码

movzx eax, byte [rax+1018h] 从地址 rax 开始 1 字节并零扩展²⁹ *(_BYTE *) (rax + 0x1018

mov 和lea 还是分岭

mov 真的读写lea 只算地址看到lea 就知道要拿这地址去用(参存指针做字符串),而不是里面内容 这一条帮你区分取指针和取指针的值。

5.8 完整运行程序: 自己造一个同样的结构

最好的方式是自己写一份代码让编译器来验证

```
e xp1 4.c — — 用真实的重量代码

#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <stddef.h>

#define NAME_AREA 0x1000

typedef struct {
    uint8_t a[0x10];
    uint8_t flag_or;
} SubObject;

typedef struct {
    uint8_t a[0x10];
    char name[NAME_AREA];
    SubObject sub;
} MainObject;

int main(void)
{
    printf("offset of (name) of f= %d\n", offsetof(MainObject, name));
    printf("offset of (sub) of f= %d\n", offsetof(MainObject, sub));

    struct SubObject s;
    struct MainObject m;

    s.flag_or = 0x5A5A;
    s.sub.a[0] = 's';

    strcpy(m.name, "reverse engineering");

    /* ????"?????"???????? */
    uint8_t *v = (uint8_t*)(*(uint64_t*)((char*)&s[0x20+10*10];18);
    const char *s = (const char*)((char*)&s[0x10];10);

    printf("tvf = 0x%02X\n", v);
    printf("tsf = %s\n", s);
    printf("tofbj.sub -> flag_or, sub.a-bf1=ag_xo%r);_2sXt\ant"e);
    return 0;
}
```

为什么这种编译方式会有问题

讲开是 $a[1] + 4112$ (0x 1010) 取字符串、 $a[1] + 8208$ (0x 2010) 取指针。本列为用数表
达前 0x 10 字节 + 0x 1000 字节区, 把 name 放在 0x 10 处 结束于 0x 1010, sub 落在
0x 2010。这说明还结构时"到底什么并不重要"偏能止、类型能解释 读写
才是唯一标准

这一章所有容易出错的地方都列出来了，建议先看问题，动脑想，再到后面的答案。

6.1 三个关键点

1. 为什么还要分类型？地不就是一个数吗？
2. `int * p;` 里的 `*` 和 `* p = 1;` 里的 `*` 是一个东西吗？
3. `p + 1` 和 `(char *)p + 1` 有什么区别？
4. 为什么 `sizeof (数组)` 和 `sizeof (数组参数)` 不一样？
5. `a[i]` 和 `*(a + i)` 完全等价，那 `2[a]` 为什么也合法？
6. 为什么 `int a[5];` 之后 `a` 不能值不能用 `a = . . .`)？
7. `&a` 和 `&a[0]` 的值一样，类型一样吗？
8. 结构体成员之间的空格里，到底能放什么？
9. 为什么结构体大小不等成员大小？
10. 为什么把成员顺序换一下，结构体就变了？
11. 为什么 `x_64` 下是 8 字节，但 `long` 在 `Windows` 上还是 4 字节？
12. `*(_QWORD *)p` 和 `*(_DWORD *)p` 在同一地址时结果一样吗？
13. 为什么编译器里，总是强制类型转换？
14. 函数的 `(*f p)()` 和 `f p()` 为什么等价？
15. 字符串为什么需要尾部的 0？没有会怎样？
16. `const char *p` 里的 `const` 到底限制谁？
17. 为什么观察到 `int` 成员出现在偏移 1？
18. `mov` 和 `lea` 的核心区别是什么？
19. 为什么逆时要从进制的偏移转换为十进制？
20. 还原结构体时，成员名重要，还是偏移重要？

6.2 逐题解答

Q1 为什么还要分类型

因为类型决定了读几个字节和 `+ 1` 走多远。0x 1000 这个地址用 `char*` 读到 1 字节，用 `int*` 读到 4 字节，同一个地址，不同类型的解释，结果可以完全不同。

Q2 声明的 * 与表达式中的 *

不是同一个东西。声明里的 `*` 属于类型的一部分（`int *` 是指针类型）；表达式里的 `*` 是一元运算符，解引用。判断方法看这个 `*` 前面是不是一个类型名。

Q3 p+1 与 (char*)p+1

如果p是int*, p+1走4字节而(char*)p+1走1字节 前者是下一个int",后者是下一个字节。这就是为什么扫描要用char*, 读整要用int*。

Q4 sizeof 数组与参数

数组在传参时会退化指针信息丢失 所以sizeof 在函数内是无效的。这就是为什么C函数必须传一个长度参数

Q5 2[a] 为什么合法

因为a[i] 定义就是*(a+i),而满足交换律*(a+i)等于*(i+a),写成标式就是i[a]。法不要这么写 它是帮助理解本质

Q6 数组为什么有值

数组在多数场合退化为指向首元素的指针,但它本身不是一块内存变量,它是数组对象的标识 所以a = p; 编译不过而char*q = a; 可以

Q7 &a 与 &a[0]

数值同都是首地址,但类型不同&a[0]是int*,&a是int(*)[5] (指整数组的指针)。因此&a + 1会跳20字节整数组,而&a[0] + 1只跳4字节 这是计算里最容易出错地方之一。

Q8 空是什么意思

是初始化的字节内容不确定可能残留上一次的栈数据 读在C里不是定义为对unsigned char而言,但它的值没有意义。逆向不要解释padding的内容

Q9 为什么大不等于成员之和

因为齐 每成员必须在对齐整数偏移上,编译器会填充字节 结构体尾要补齐到最大对齐的数

Q10 交换顺序为什么变少

把成员前面小成员放在一起可以减少产生空阔 经验则按齐值从小到大排列成员

Q11 为什么Windows上long是4字节

因为Windows使用LLP64模型(long long 和指针是64位 long 仍是32位,而Linux(long 也是64位。这直接影响了结构体布局是平台必须明确事实

❶ Q1 2 _QWORD 与 _DWORD 读地址

不一样 _QWORD 读8 字节 _DWORD 读4 字节 如果位置实际是8 字节用 _DWORD 读会得到4 字节在x 64 上会得到一个错误的地址 所以会判断错会导致析构

❷ Q1 3 为什么是转

为改变读取方式和解码效率。编译器优化后统一的数据类型需要统一转成 _DWORD *,需要字节转成 _BYTE *,需要字符串转成 char*。

❸ Q1 4 (*f p)() 与 f p()

两者等价。函数运算符() 的左需要是一个函数指针 f p 本身就是(*f p) 解引用是一个地址 编译器接受两种写法是为了提醒读者 f p 是指针。

❹ Q1 5 字符串为什么需要 \0

因为C 字符串不保长度所有字符串函数都读到0x 00 来结束 如果没有0,strlen 会一直往后读直到内存的某个0,轻则结果错误,重则崩溃 这是常见漏洞之一。

❺ Q1 6 const char *p 限制

限制的是通过p 修改它指向的内容,不限制p 自身指向 要限制p 自身需要写成 const char *p。

❻ Q1 7 为什么 int 成员偏移1

几乎可以断定结构体用了#pragma pack (1) 或__attribute__((packed)) 游戏引擎在IDA 中可以给结构体设置packed 属性匹配

❼ Q1 8 mov 与 lea

mov 讲内存读写数据,lea 只算地址不讲内存 在这里,lea 常出现在准备地址给函数或取字符串指针的位置

❽ Q1 9 为什么换十六进制

因为结构体偏移由2 的幂次决定在十六进制下规律明显 0x 10、0x 18、0x 20、0x 1010、0x 2010 之间的倍数一眼可见,十进制则完全看不出

❾ Q20 成员名还是偏移

偏移决定准确性 名字可读性 一个结构体只有正确的偏移与类型 哪成员叫field_0、field_4,也能正确解释 程序为 随理解深再改名可

6.3 动手练习

下面题请先算出答案再看答案

练习 1

在 64 位 Windows 上, `sizeof(struct S)` 是多少? 各成员偏移多少?

```
struct {
    char a;
    int b;
    char c;
    long d;
};
```

练习 2

下面代码输出什么? 先不要编译

```
int a[3] = { 1, 2, 3 };
int *p = a + 1;
printf("%d\n", *p[-1]);
```

练习 3

已知 `a` 是结构体, 写出等价的宏定义

```
#define a 1
#define a 2
#define a 3
```

练习 4

把上面结构体定义成 `typedef` 定义, 并说明成员类型与偏移

练习 5

为什么下面代码在 64 位程序里是危险的?

```
int *p = (int *)SomePos;
int *q = *(int *)((char *)p + 1);
```

6.4 练习答案

答案1

```
??      ??      ??      ??? ?      ????
-----
a          1          1          ?? 0
b          4          4          1 ?? 4 ??? -> ?? 4          4
c          1          1          8
d          8          8          9 ?? 8 ??? -> ?? 16          16
-----
???? 24, ???? 8, 24 ?? 8 ???
sizeof(struct S) = 24          (???? 14 ??, ?? 10 ??)
```

答案2

输出"2 1"。p 指向 a [1] , 所以 * p 是 2 ; p [- 1] 等于 * (p - 1) , 即 a [0] , 是 1 。负标是合法的写法

答案3

```
/* ?? */
*( _ D W O *) ( 0 1 + 1 0 ) = 1 0 ; 0
*( _ B Y T ) ( *( _ Q W O *) ( 0 1 + 3 2 ) + 8 ) = 1 ;

/* ?????? ( a 1 ? SomeStruct * ) */
a 1 > f i e l d _ 1 0 = 0 / 0 * 0 ? 8 ???? ? * /
a 1 > f i e l d _ 3 2 = 1 ; 8 / * + 0 x 2 0 ???? , ???? + 0 x 8 ?? 1 ?? */
```

答案4

```
typedef u_cIt n{e r
    u n s i g n e a d r n k n o w [ 8 ] ; 0 0 / 0 * 0 ? 8 ???? ? * /
    u n s i g n e a d r n k n o w [ 0 ] ; 0 1 / 4 * + 0 x 1 4 ~ + 0 x 1 F ( ? ? ? 8 ? ? ? ? ? ) * /
    / * . . . */
} I n n e r ;

typedef u_cO t t{e r
    u n s i g n e a d r n k n o w [ 0 ] ; 0 0 / 0 * 0 + 0 x 0 ~ + 0 x F ? ? ? * /
    i n t f i e l d _ 1 6 / * + 0 x 1 0 ? ? ? 1 0 0 , ? 4 ? ? * /
    u n s i g n e a d r n k n o w [ 0 ] ; 0 1 / 4 * + 0 x 1 4 ~ + 0 x 1 F ( ? ? ? 8 ? ? ? ? ? ) * /
    I n n e r * f i e l d _ 3 2 / * + 0 x 2 0 8 ? ? ? ? ( x 6 4 ) * /
} O u t e r ;

/* ?? : o f f s e t o f ( O u t e r , f i e l d _ 1 6 ) == 0 x 1 0
    o f f s e t o f ( f i e l d _ 1 6 ) == 0 x 2 0
    o f f s e t o f ( I n n e r , f i e l d _ 3 2 ) == 0 x 8 */
```

注意 f i e l d _ 3 2 是 8 字节指针的偏移，必须是 8 的倍数，所以 0x 10 + 4 = 0x 14 之后需要到 0x 18 直接到 0x 20 ; 具体是 0x 18 还是 0x 20 取决于之前有没有成员，这说明只有结合代码

才能不能凭空

I 答案5

(c h a r *)p + 1 只进1 字节得到的地址通常不是4 的倍数并取i nt。在x 8 6 上可能跑变慢,在A R M 等架构上会异常而且从语义上讲跨1 字节读一个i nt,读的是上一个i nt 的后3 字节+ 下一个i nt 的第1 字节的接片肯定不是想要的结果

这题意义

它考察了计算量纲对变和跨型取三个知识点能把这一题讲清楚说明的内容基本吃透了。

I 6 . 5 学习建议

- 1 把第0 章三个实验跑一遍确保输出与预期一致
2. 把第3 章的e x p7 改一改把i nt 换成d o u b l e 、s h o r t,观察+ 1 的偏移
3. 把第4 章的e x p12 改为你自己的一个结构用o f f s e t o f 验证你的值
4. 找一个真的程序(再写一个带结构的,用调试器看结构体在内存里的样子)
5. 用I D A F r e e 或G h i d r a 打它把反编译出的偏移与你结构体对照
- 6 回第5 章把目标代码的结构体换成你自己的去看注释通
7. 最后找一道C T F 逆题(俺的c r a c k m e),尝试其中一个结构体

I 6 . 6 相对表

收	效	说明
解用	d e r e f e r e n c e	通过访问指针的内存
取址	a d d r e s s - o f	用&得到一个变量的地址
字序	e n d i a n n e s s	多字节数据的排列顺序
小端	l i t t l e - e n d i	低位字节在低地址
大端	b i g - e n d i a n	高位字节在低地址
对齐	a l i g n m e n t	数据地址必须是某值的整数倍
填充	p a d d i n g	为对齐而填充的字节
空洞	h o l e / s l a c k	填充造成的未使用空间
退化	d e c a y	数组长表达式变成指针
虚表	v t a b l e	虚函数的函数指针数组
偏移	o f f s e t	相对于基址的字节距离

收	效	说明
强制类型转换	c a s t	^{3 7} 改变编译时某地址的解释方式

收	效	说明
---	---	----

6.7 一句总结

把本章往一句总结

结构就是一个数字 + 一个类型：数字是地址，类型决定多宽、走多远。结构就是若干成员按齐规则排开，偏移量成员的大小与对齐决定。逆的排列作就是这规则反过来用。从成员里的偏移量倒推出成员类型和顺序。